جامعة عين شمس
كلية حاسبات و معلومات

Project Allocation

Project Title:

FixMate

Outperforming the Teachers with Multi-LLM Distillation and RAG

Department: Scientific Computing

Supervisors:

No	Name	Department
1	**Dr. Dina El Sayad**	**Scientific Computing**
2	**TA. Aya Nasser**	**Scientific Computing**

Students Names:

No	Name	Department	Email
1	محمد مدحت عبدالعليم عبده	Scientific Computing	2022170388@cis.asu.edu.eg
2	ياسر رضا محمد رجب	Scientific Computing	2022170489@cis.asu.edu.eg
3	اسلام احمد عادل محمدعبد الحليم	Scientific Computing	2022170057@cis.asu.edu.eg
4	إسراء طه محمود علي سعيد	Scientific Computing	2022170600@cis.asu.edu.eg
5	جهاد محمود ادم سيد	Scientific Computing	2022170113@cis.asu.edu.eg
6	عمر خالد محمد عبد الوهاب	Scientific Computing	2022170276@cis.asu.edu.eg

Team Leader Number: 01014487283

Introduction:

Software testing is a critical step in ensuring the reliability and robustness of applications. Traditional manual test case design is time-consuming, error-prone, and struggles to keep up with the complexity of modern software projects. Recent advances in large language models (LLMs) demonstrate strong capabilities in code understanding, generation, and repair, offering new opportunities for automating software testing and bug fixing.

Our project proposes a system that leverages LLMs to automatically generate, refine, and optimize test cases, while also incorporating bug fixing capabilities. By distilling knowledge from multiple teacher LLMs (DeepSeek-V3, Qwen 3.5-Coder, Llama 3.1 405B variant, Mistral Large 2) into a lightweight student model (Qwen 3.5 Coder 7B) and enhancing it with Retrieval-Augmented Generation (RAG), we aim to create a scalable, memory-efficient, and accurate pipeline. Additionally, the system includes bug fixing inspired by APR, coverage heatmaps for visual analysis, and Control Flow Graphs (CFGs) to map test paths, providing developers with actionable insights.

Problem Definition:

- Manual test-case creation requires expert knowledge, making it costly and slow.
- Existing LLMs (e.g., GPT, Claude) can generate strong tests but are expensive to run and inaccessible for continuous integration.
- A single teacher model often introduces noise or inconsistency in generated test cases.
- Generated tests may pass execution but fail to cover enough code paths or detect subtle faults.
- Bug fixing is often manual or reliant on incomplete tools, lacking integration with test generation.
- Lack of visual aids like heatmaps and CFGs makes it hard to understand coverage and test paths.

These limitations motivate the need for a distilled, lightweight student model that combines the strengths of multiple LLMs while retaining efficiency and scalability, with added bug fixing and visualization features.

Motivation:

- Reduce reliance on paid API-based LLMs by building an open, distilled model.
- Improve test quality and coverage by combining knowledge from multiple teacher LLMs.
- Integrate memory through RAG to reuse validated test cases across users and projects.
- Provide a research contribution: showing that voting + distillation + RAG can outperform single-teacher models.

Objectives:

- Develop a pipeline to automatically generate test cases from source code (Python/GitHub).
- Apply multi-teacher distillation into a student model (Qwen 3.5 Coder 7B) with a voting and filtering mechanism.
- Build a Meta-Student by training multiple students and merging their outputs.
- Implement RAG memory to store and reuse validated test cases globally and per user.
- Integrate mutation testing and coverage analysis to evaluate and refine test quality.
- Deliver a usable system prototype with a simple interface for developers.
- Add bug fixing capabilities using APR techniques.
- Generate coverage heatmaps to visualize test effectiveness.
- Produce Control Flow Graphs (CFGs) for analyzing test paths.
- Incorporate dependency graphs for dependency analysis in testing and bug fixing.

Time Plan:

	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May
Literature Review, Dataset collection								
Teacher model selection and initial test case generation								
				FINAL				
Multi-teacher distillation (students + Meta-Student)								
RAG integration (global + per-user)								
Mutation testing & coverage-based evaluation								
Iterative refinement system & HiTL integration								
System integration, documentation								

Tools:

- Programming Languages: Python
- Libraries: Hugging Face Transformers, LangChain/LangGraph, spaCy (NER/Parsing), Pytest, Coverage.py, Mutmut (mutation testing)
- AI Models: Teachers (DeepSeek-V3, Qwen 3.5-Coder, Llama 3.1 405B, Mistral Large 2)
- Vector Store: FAISS / Weaviate (for RAG)
- Development Environment: Jupyter, VSCode, Docker for sandboxing tests
- Additional Tools for Bug Fixing: PyFix or similar APR libraries
- For Coverage Heatmaps: Matplotlib, Seaborn
- For CFGs: Radon, Lizard
- For Dependency Graphs: NetworkX, PyDepGraph

References:

- Iterative Knowledge Distillation through Feedback-Driven Learning Cycles (Chen et al., 2024)
- Automated Extraction of Research Software Installation Instructions from README Files: An Initial Analysis
- A Survey of Learning-based Automated Program Repair (Quanjun Zhang et al., 2023)
- An Empirical Study on LLM-based Agents for Automated Bug Fixing (Xiangxin Meng et al., 2024)
- A Survey on Automated Program Repair Techniques (K. Huang et al., 2023)
- Containerization and its Architectures: A Study Singh, A., & Sharma, A. (2023). Containerization and its Architectures: A Study. ResearchGate.
- An Introduction to Docker and Analysis of its Performance Soni, A., & Kalra, R. (2017). An Introduction to Docker and Analysis of its Performance. ResearchGate.
- Graph-based RAG Enhancement via Global Query Disambiguation and Dependency-Aware Reranking" by Ningyuan Li et al. (2025)
- You Don't Need Pre-built Graphs for RAG" (2025) by S. Chen

- Retrieval-Augmented Generation for Large Language Models: A Survey" by Yunfan Gao et al. (2023)
- Retrieval-Augmented Generation for AI-Generated Content: A Survey" by Penghao Zhao et al. (2024)